



Construcción de aplicaciones web en tiempo real con Node.js

Breve descripción:

Configurar un entorno profesional, levantar un servidor con Node.js y establecer canales de comunicación bidireccional mediante WebSockets son los pasos que separan el conocimiento teórico de una aplicación funcional. Este componente aborda la construcción práctica de soluciones web en tiempo real, desde la infraestructura hasta la integración de sockets, con criterio técnico y código estructurado.

Agosto 2026

Tabla de contenido

Introducción	3
1. Preparación del entorno de desarrollo	6
1.1. Requerimientos de hardware y software	6
1.2. Entorno de desarrollo integrado y configuración inicial	8
2. Construcción del servidor con Node.js.....	12
2.1. Arquitectura del servidor básico	12
2.2 WebSockets: comunicación bidireccional en tiempo real.....	14
3. Aplicación práctica: pizarra virtual colaborativa.....	21
3.1 Estructura del proyecto	21
3.2 Código del servidor.....	22
3.3 Código del cliente	24
4. Flujo lógico del desarrollador	28
Síntesis	30
Glosario	32
Referencias bibliográficas	35
Créditos	36

Introducción

Un usuario escribe un mensaje y en décimas de segundo aparece en la pantalla de otra persona al otro lado del país. Detrás de esa experiencia existe una arquitectura técnica precisa: un servidor activo, un canal de comunicación persistente y código que responde en tiempo real. Construir esa arquitectura es exactamente el propósito de este recorrido.

El camino avanza desde la configuración profesional del entorno de desarrollo hasta la construcción de un servidor funcional con Node.js y la integración de sockets para establecer canales de comunicación bidireccional. Cada concepto se aborda desde su funcionamiento interno para poder anticipar fallos y tomar decisiones técnicas argumentadas frente a escenarios reales.

El punto de llegada es una aplicación web funcional donde múltiples clientes se comunican de forma simultánea a través del servidor, no como ejercicio académico aislado, sino como demostración concreta de las mismas tecnologías que impulsan los sistemas de chat corporativos, tableros financieros en vivo y plataformas de colaboración más exigentes del mercado actual.

Video 1. Construcción de aplicaciones web en tiempo real con Node.js



[Enlace de reproducción del video](#)

Transcripción del video: construcción de aplicaciones web en tiempo real con Node.js

Construcción de aplicaciones web en tiempo real con Node JS. Un usuario escribe un mensaje y en décimas de segundo aparece en la pantalla de otra persona al otro lado del país. Detrás de esa experiencia existe una arquitectura técnica precisa, un servidor activo, un canal de comunicación persistente y un código que responde en tiempo real.

Construir esa arquitectura es exactamente el propósito de este recorrido. El recorrido inicia con la configuración profesional del entorno de desarrollo, los requerimientos de hardware y software, el uso del entorno de desarrollo integrado VS Code y la inicialización del proyecto. A partir de esa base se avanza hacia la construcción de un servidor funcional con Node JS Express, comprendiendo su arquitectura interna y las razones detrás de cada decisión de diseño.

Transcripción del video: construcción de aplicaciones web en tiempo real con Node js

Posteriormente se aborda la integración de Sockets mediante Sockets. Para establecer canales de comunicación bidireccional en tiempo real, superando las limitaciones del protocolo HTTP tradicional. Cada concepto se trabaja desde su funcionamiento interno para poder anticipar fallos y tomar decisiones técnicas argumentadas.

El punto de llegada es una aplicación web funcional, una pizarra virtual colaborativa donde múltiples clientes interactúan de forma simultánea a través del servidor, no como ejercicio aislado, sino como demostración concreta de las mismas tecnologías que impulsan los sistemas de chat corporativos, los tableros financieros en vivo y las plataformas de colaboración más exigentes del mercado actual. Oh.

1. Preparación del entorno de desarrollo

La construcción de software no es un acto mecánico: es un proceso intelectual e iterativo. En el desarrollo web en tiempo real, los datos no llegan en orden predecible; programar para este escenario exige un pensamiento sistémico y reactivo, capaz de anticipar errores, leer mensajes de la consola y trazar la ruta del dato desde el navegador hasta el servidor.

El primer paso para lograrlo es configurar un entorno de desarrollo profesional. Un entorno mal configurado es la principal fuente de errores difíciles de rastrear y la causa más común de frustración en etapas tempranas del desarrollo.

1.1. Requerimientos de hardware y software

Requerimientos mínimos del entorno de desarrollo profesional

Para garantizar un entorno de desarrollo estable, se deben considerar dos dimensiones: los componentes físicos (hardware) y el software base del sistema.

- **Requerimientos de hardware**

- **Memoria de Acceso Aleatorio (RAM)**

Aunque Node.js consume pocos megabytes por sí solo, el entorno completo de desarrollo demanda considerablemente más recursos. Al tener activo simultáneamente el editor de código, el navegador web con varias pestañas, la consola ejecutando el servidor y un cliente de base de datos, el consumo de RAM se incrementa de forma significativa. El estándar mínimo recomendado es de 8 GB, siendo 16 GB la configuración ideal para trabajar sin latencia. Un equipo con menos de 4 GB de RAM

producirá cierres inesperados del servidor y tiempos de respuesta inaceptables durante el desarrollo.

- **Procesador (Unidad Central de Procesamiento, CPU)**

Se recomienda un procesador multinúcleo moderno, por ejemplo, Intel Core i5/i7 o AMD Ryzen 5 o superior. La indexación de archivos del proyecto y la transpilación de código requieren una capacidad de procesamiento adecuada. Un procesador de generación anterior ralentizará la detección de errores en tiempo real y afectará la fluidez del entorno de desarrollo integrado.

- **Almacenamiento (Unidad de Estado Sólido, SSD)**

La carpeta `node_modules` de un proyecto `Node.js` puede contener miles de microarchivos. La diferencia de velocidad entre una Unidad de Estado Sólido (SSD) y un disco duro tradicional (HDD) para leer y escribir estos archivos es determinante: un HDD puede tardar varios minutos en instalar dependencias que un SSD resuelve en segundos. Trabajar con HDD en proyectos `Node.js` de mediana escala genera tiempos de instalación prolongados y errores intermitentes de lectura.

- **Requerimientos de software**

- **Sistema operativo**

`Node.js` es multiplataforma y funciona en Windows, macOS y Linux. Sin embargo, en la industria de servidores, Linux es el estándar de facto. Para quienes trabajan en Windows, instalar Windows Subsystem for Linux (WSL) permite acceder a una terminal Bash real sin abandonar el sistema operativo principal, una práctica ampliamente adoptada en equipos de

desarrollo profesional. Familiarizarse con la terminal desde etapas tempranas reduce significativamente la curva de aprendizaje al momento de desplegar aplicaciones en servidores de producción.

- **Node.js y el gestor de paquetes de Node (Node Package Manager, npm)**

Deben estar correctamente instalados y verificados antes de iniciar cualquier proyecto. La instalación se cubre en detalle en la sección 1.2. Intentar construir una aplicación sin verificar previamente el entorno produce errores de compatibilidad que son difíciles de diagnosticar si no se conoce el estado exacto de las herramientas instaladas.

1.2. Entorno de desarrollo integrado y configuración inicial

Un entorno de desarrollo integrado (Integrated Development Environment, IDE) no es un simple editor de texto: es una plataforma que centraliza la escritura, ejecución y depuración del código en una sola interfaz. En el ecosistema Node.js, el IDE de referencia en la industria es Visual Studio Code (VS Code).

VS Code está construido sobre Electron, un framework que permite ejecutar aplicaciones web dentro de una instancia incrustada del navegador Chromium. Esto explica por qué funciona de forma idéntica en cualquier sistema operativo y por qué consume una cantidad de memoria RAM considerable. A cambio, ofrece tres funcionalidades críticas para el desarrollo profesional:

Resaltado de sintaxis

Diferencia visualmente palabras clave, variables y funciones mediante color, reduciendo la carga cognitiva al leer el código.

IntelliSense

Muestra en tiempo real la documentación de las funciones de Node.js disponibles, acelerando la escritura y reduciendo errores tipográficos.

Terminal integrada

Permite ejecutar comandos sin salir del editor, unificando la escritura y la ejecución en una sola pantalla.

- **Configuración inicial del proyecto**

Una vez instalado el IDE, se configura el entorno siguiendo estos pasos:

- a) Verificación del entorno**

Antes de construir, se verifica que las herramientas estén disponibles en el sistema. En la terminal integrada de VS Code se ejecutan los siguientes comandos:

Bash

```
node --version
```

```
npm --version
```

Si ambos comandos retornan un número de versión, el entorno está operativo. Si alguno produce un error, la instalación de Node.js debe revisarse antes de continuar; intentar avanzar sin esta verificación generará errores de módulo no encontrado imposibles de rastrear en etapas posteriores.

b) Creación del directorio del proyecto

Se crea una carpeta vacía que actuará como el entorno aislado del proyecto. Toda la lógica, las dependencias y los archivos de configuración residirán dentro de este directorio. Mezclar proyectos en una misma carpeta sin estructura genera conflictos de versiones y dificulta el trabajo en equipo.

c) Inicialización del proyecto

Desde la terminal, con la ruta apuntando a la carpeta del proyecto, se ejecuta:

```
bash
```

```
npm init -y
```

Este comando genera automáticamente el archivo **package.json**, el documento de identidad del proyecto. Registra el nombre del sistema, la versión, el archivo principal de entrada (**index.js**) y el inventario de todas las dependencias de terceros que se instalarán. La flag -y confirma automáticamente todos los valores por defecto, agilizando la inicialización. Si el **package.json** se elimina accidentalmente de un proyecto avanzado, se pierde el registro exacto de las dependencias instaladas. Otro desarrollador que intente ejecutar el proyecto no podrá reconstruir el entorno correctamente, comprometiendo la portabilidad y el mantenimiento de la aplicación.

d) Instalación de dependencias

Con el **package.json** generado, se instalan las bibliotecas necesarias para construir el servidor y la capa de sockets:

```
bash
```

```
npm install express socket.io
```

Este comando descarga los paquetes en la carpeta **node_modules** y registra las versiones exactas en **package.json** y **package-lock.json**. La carpeta **node_modules** nunca se comparte ni se sube al repositorio de control de versiones, solo se comparte el **package.json**, que contiene toda la información necesaria para reconstruirla con un solo comando.

2. Construcción del servidor con Node.js

Con el entorno configurado y las dependencias instaladas, el siguiente paso es levantar el servidor, la columna estructural de toda aplicación web en tiempo real. El servidor es el ente que permanece activo, escucha las peticiones de los clientes y coordina el flujo de información entre ellos.

2.1. Arquitectura del servidor básico

Node.js por sí solo no impone una estructura para gestionar rutas ni peticiones HTTP. En la industria, esa responsabilidad recae sobre el framework Express, una capa ligera que organiza la lógica del servidor sin agregar complejidad innecesaria al proyecto.

- **Arquitectura básica de un servidor Node.js con Express**

Para que una aplicación web exista, un servidor debe estar activo y disponible para recibir peticiones de clientes o navegadores. Aunque Node.js incluye un módulo nativo llamado http, en la industria profesional se utiliza el framework Express, que simplifica la creación de rutas y la gestión del servidor sin agregar complejidad innecesaria.

Un servidor HTTP tradicional funciona como el recepcionista de un hotel muy concurrido. Cuando llega un huésped (el cliente o navegador web), realiza una petición específica, por ejemplo, solicitar la llave de una habitación (petición HTTP GET). El recepcionista procesa la solicitud, entrega la llave (respuesta HTTP) y de inmediato olvida al huésped. Si este regresa cinco minutos después a pedir una toalla, debe identificarse desde cero. A esta ausencia de memoria se le denomina protocolo stateless (sin estado): eficiente para páginas estáticas, pero completamente

inadecuado para aplicaciones en tiempo real donde el servidor debe recordar quién está conectado y qué está haciendo.

Antes de escribir el código, se instalan las herramientas necesarias desde la terminal:

```
bash  
npm install express socket.io
```

A continuación, se crea el archivo **index.js**, el punto de entrada de la aplicación:

a) Importación de módulos

Se cargan los módulos **express** y **http** en la RAM del servidor. El módulo **http** nativo es necesario porque más adelante se le inyectará la capa de sockets; Express por sí solo no soporta WebSockets.

```
javascript  
const express = require('express');  
const http = require('http');
```

b) Inicialización de la aplicación

Al ejecutar **express()** se crea una instancia de la aplicación.

Luego, **http.createServer(app)** construye un servidor físico de red que delega la inteligencia de rutas a Express. Esta separación es deliberada: permite que el mismo servidor atienda tanto peticiones HTTP tradicionales como conexiones WebSocket persistentes.

```
javascript  
const app = express();  
const servidor = http.createServer(app);
```

c) Definición de una ruta básica

Se define la respuesta a la ruta raíz usando req (Request - la petición entrante) y res (Response - la respuesta saliente). Si el servidor no tiene ninguna ruta definida, cualquier petición producirá un error 404.

javascript

```
app.get('/', (req, res) => {  
  res.send('Servidor en línea y esperando conexiones.');
```

```
});
```

d) Puesta en marcha del servidor

servidor.listen bloquea el puerto 3000 y mantiene el proceso activo indefinidamente dentro del event loop. Si el puerto ya está en uso por otra aplicación, Node.js arrojará un error **EADDRINUSE** — en ese caso se debe detener el proceso que ocupa ese puerto o cambiar el número de puerto en la variable **PUERTO**.

javascript

```
const PUERTO = 3000;  
servidor.listen(`PUERTO`, () => {  
  console.log(`Servidor ejecutándose en http://localhost:${PUERTO}`);  
});
```

2.2 WebSockets: comunicación bidireccional en tiempo real

El servidor HTTP construido en la sección anterior resuelve eficientemente las peticiones tradicionales: un cliente pregunta, el servidor responde y la conexión se cierra. Sin embargo, este modelo tiene un límite estructural que lo hace inviable para

aplicaciones en tiempo real: el servidor jamás puede iniciar una conversación; siempre espera a que el cliente dé el primer paso.

Tabla 1. Comparación entre HTTP tradicional (polling) y WebSockets

Característica	HTTP tradicional (polling)	WebSockets
Tipo de conexión	Nueva conexión por petición	Conexión persistente
Quién inicia	Solo el cliente	Cliente o servidor
Dirección	Unidireccional	Bidireccional (full-duplex)
Consumo de red	Alto - peticiones repetitivas	Bajo - canal único abierto
Uso ideal	Páginas estáticas	Tiempo real: chat, tableros, GPS

Para simular tiempo real sobre HTTP, en el pasado se usaba una técnica llamada short polling (sondeo corto): programar el navegador para enviar una petición automática cada pocos segundos preguntando: "¿hay mensajes nuevos?". El resultado es el equivalente informático a alguien preguntando repetidamente si ya llegó un correo; la mayoría de las veces la respuesta es negativa, pero el servidor igual debe procesar cada consulta, desperdiciando ancho de banda y saturando la red.

WebSockets resuelve este problema de raíz estableciendo un canal de comunicación persistente, bidireccional y full-duplex: tanto el cliente como el servidor pueden enviar y recibir información en cualquier momento, sin necesidad de nuevas peticiones.

Para comprender de forma conversacional cómo WebSockets supera las limitaciones del protocolo HTTP tradicional, se sugiere explorar el siguiente recurso de audio:

Transcripción del pódcast. Del polling al tiempo real: cómo funcionan los WebSockets

Hombre: del polling al tiempo real, ¿cómo funcionan los WebSockets? ¿Alguna vez se han preguntado qué hay detrás de esa sensación de inmediatez cuando usan un chat o ven datos cambiar en vivo? Hoy vamos a descubrirlo.

Mujer: y es que esa magia, aunque tiene un nombre que parece sacado de la NASA, en realidad tiene una lógica muy sencilla: se trata de dejar de pedir permiso para recibir información y empezar a recibirla en el momento exacto en que ocurre.

Hombre: porque lo que hacíamos antes con el protocolo HTTP tradicional era igual que enviar un correo físico y sentarse a esperar la respuesta. Si queríamos saber si había algo nuevo, nos tocaba ir hasta el buzón y revisar manualmente.

Mujer: esa es la famosa técnica del short polling. Pero hoy día eso es un proceso tan ineficiente como estar preguntando cada 5 minutos si ya llegó el paquete, en lugar de que nos avisen apenas llegue.

Hombre: y claro, eso agota a cualquiera. Imaginen a miles de usuarios haciendo esa misma pregunta al tiempo. El servidor termina saturado procesando consultas donde la respuesta casi siempre es un "todavía no hay nada nuevo".

Mujer: aquí es donde los WebSockets rompen ese ciclo. En lugar de estar preguntando, establecen un intercomunicador abierto. Se pide un ascenso de conexión y el canal queda activo de forma permanente.

Hombre: es comunicación total en ambas direcciones. Ya el servidor no tiene que esperar que le envíen una petición, sino que ya él puede enviar los datos en el instante en que suceden, como un impulso eléctrico.

Mujer: eso sí es pasar al siguiente nivel. Dejamos atrás el formato de pregunta y respuesta y pasamos a tener un flujo de datos constante, sin cabeceras pesadas ni esperas innecesarias.

Hombre: por eso esta herramienta es la base de todo lo que llamamos "en vivo", como chats, tableros financieros, juegos multijugador o cualquier sistema donde la información cambia cada segundo.

Mujer: simplemente es conectar el mundo al ritmo en que suceden las cosas.

Hombre: dominar este concepto es lo que separa al programador aficionado del experto que desarrolla aplicaciones para el mundo real.

Mujer: si quieren ver cómo se traduce esto a código real, los invitamos a revisar el material de apoyo.

Hombre: gracias por seguir conectados con nosotros. ¡Nos escuchamos en el próximo programa!

- **El proceso de conexión**

- **Momento 1**

- La petición inicial**

- El navegador realiza una petición HTTP normal, pero incluye una cabecera especial solicitando un upgrade (mejora de conexión). Es como solicitar pasar de una carta postal a una llamada telefónica directa.

- **Momento 2**

- La aceptación del servidor**

- Si el servidor está configurado para WebSockets, acepta la solicitud y confirma el upgrade. A partir de este instante la conexión HTTP se transforma en un túnel TCP abierto de forma permanente.

- **Momento 3**

- El canal activo**

- El canal queda abierto indefinidamente. El servidor adquiere la capacidad de enviar (push) datos al cliente en el momento exacto en que ocurren, sin que el cliente tenga que preguntar. Este es el principio de la comunicación en tiempo real.

- **Integración de sockets al servidor**

Se modifica el archivo index.js para agregar la capa de WebSockets mediante socket.io.

A continuación, se describe el análisis crítico de cada bloque:

javascript

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const app = express();
const servidor = http.createServer(app);
const io = new Server(servidor);
io.on('connection', (socket) => {
    console.log(`Nueva conexión establecida. ID: ${socket.id}`);
    socket.on('mensaje_chat', (datos) => {
        console.log('Mensaje recibido:', datos);
        io.emit('mensaje_chat', datos);
    });
    socket.on('disconnect', () => {
```

```
console.log(`Conexión cerrada. ID: ${socket.id}`);  
});  
});  
servidor.listen(3000, () => {  
  console.log(`Servidor con sockets ejecutándose en el puerto 3000.`);  
});
```

- **Const io = new Server(servidor)**

Se crea el puente entre el mundo HTTP tradicional y el mundo de los WebSockets. La variable io representa el centro de control de todas las comunicaciones en tiempo real. Sin este paso, el servidor solo puede atender peticiones HTTP convencionales y no tiene capacidad de mantener conexiones persistentes.

- **io.on('connection', (socket) =>**

Este es el vigilante de la puerta: cada vez que un navegador establece conexión, se dispara este evento. El objeto socket representa a ese usuario específico. Si se conectan 50 clientes simultáneamente, habrá 50 objetos socket distintos en memoria, cada uno identificado con un socket.id único. No gestionar correctamente la desconexión mediante el evento disconnect provoca que estos objetos permanezcan en memoria indefinidamente, generando una fuga de memoria (memory leak) que degrada el rendimiento del servidor con el tiempo.

- **Socket.on vs. io.emit()**

Esta distinción es fundamental en la arquitectura de aplicaciones en tiempo real:

- **socket.on():** escucha eventos de un usuario específico.
- **socket.emit():** envía un mensaje únicamente al emisor.
- **socket.broadcast.emit():** envía a todos los conectados excepto al emisor.
- **io.emit():** funciona como un megáfono: transmite el mensaje a todos los clientes conectados simultáneamente.

Elegir incorrectamente entre estas cuatro opciones produce errores de lógica difíciles de detectar: mensajes que no llegan, mensajes duplicados o usuarios que reciben información que no les corresponde.

Tabla 2. Métodos de emisión en socket.io y alcance por destinatario

Método	¿A quién llega?
socket.emit()	Solo al emisor
socket.on()	Escucha eventos de un cliente específico
socket.broadcast.emit()	A todos los clientes conectados excepto al emisor
io.emit()	A todos los clientes conectados sin excepción

3. Aplicación práctica: pizarra virtual colaborativa

Los conceptos abordados hasta este punto (servidor HTTP, WebSockets y emisión de eventos) adquieren sentido real cuando se integran en una aplicación funcional. Una pizarra virtual colaborativa es el escenario ideal para ello: múltiples usuarios conectados simultáneamente pueden dibujar en tiempo real y cada trazo aparece de forma instantánea en las pantallas de todos los participantes.

Esta aplicación no es un ejercicio decorativo. Comparte la arquitectura central de herramientas profesionales como Google Docs, Figma o Miro: un servidor que coordina eventos entre clientes sin almacenar el estado de la sesión en disco.

3.1 Estructura del proyecto

Antes de escribir el código, se organiza el proyecto con la siguiente estructura de archivos:

```
pizarra-virtual/  
— package.json  
— package-lock.json  
— index.js  
— public/  
  └─ index.html
```

La carpeta **public/** contendrá el archivo HTML que se entregará al navegador. El archivo **index.js** será el servidor. Esta separación entre código del servidor y archivos del cliente es una convención estándar en proyectos Node.js; mezclarlos en la misma carpeta raíz dificulta el mantenimiento y la escalabilidad del proyecto.

3.2 Código del servidor

Con la estructura del proyecto definida, se construye el archivo `index.js` que actuará como núcleo de la aplicación. Este servidor combina la entrega de archivos estáticos al navegador con la gestión de eventos de dibujo en tiempo real mediante sockets.

- **Configuración del servidor con entrega de archivos estáticos**

`express.static` le indica al servidor que entregue automáticamente cualquier archivo dentro de la carpeta **`public/`** cuando un navegador lo solicite. Sin esta línea, el servidor respondería con error 404 a cualquier petición de archivos HTML, CSS o JavaScript del cliente. El módulo nativo **`path`** garantiza que la ruta funcione correctamente en cualquier sistema operativo, independientemente de si usa barras `/` o `\` como separador.

```
javascript
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const path = require('path');
const app = express();
const servidor = http.createServer(app);
const io = new Server(servidor);
app.use(express.static(path.join(__dirname, 'public')));
```

- **Gestión de eventos de dibujo en tiempo real**

El evento **'dibujar'** recibe del cliente las coordenadas del trazo y las reenvía a todos los demás participantes usando **socket.broadcast.emit()**. Se usa **broadcast** en lugar de **io.emit()** deliberadamente: el cliente que dibuja ya ve su propio trazo en pantalla de forma local, por lo que enviárselo de vuelta generaría una duplicación visual. Este es un ejemplo concreto de por qué la distinción entre métodos de emisión tiene consecuencias directas en el comportamiento de la aplicación.

javascript

```
io.on('connection', (socket) => {  
  console.log(`Cliente conectado: ${socket.id}`);  
  socket.on('dibujar', (datos) => {  
    socket.broadcast.emit('dibujar', datos);  
  });  
  socket.on('disconnect', () => {  
    console.log(`Cliente desconectado: ${socket.id}`);  
  });  
});
```

- **Inicio del servidor**

javascript

```
const PUERTO = 3000;  
servidor.listen(PUERTO, () => {  
  console.log(`Pizarra virtual ejecutándose en http://localhost:${PUERTO}`);  
});
```

3.3 Código del cliente

El archivo **public/index.html** contiene toda la interfaz y la lógica del cliente en un solo archivo:

```
html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Pizarra virtual colaborativa</title>
  <style>
    body { margin: 0; background: #1a1a2e; display: flex;
      justify-content: center; align-items: center;
      height: 100vh; }
    canvas { background: #ffffff; cursor: crosshair;
      border-radius: 8px; }
  </style>
</head>
<body>
  <canvas id="pizarra" width="800" height="500"></canvas>
  <script src="/socket.io/socket.io.js"></script>
  <script>
    const socket = io();
    const canvas = document.getElementById('pizarra');
    const ctx = canvas.getContext('2d');
    let dibujando = false;
```

```
let xAnterior = 0;
let yAnterior = 0;
function dibujarLinea(x0, y0, x1, y1, color, emitir) {
  ctx.beginPath();
  ctx.moveTo(x0, y0);
  ctx.lineTo(x1, y1);
  ctx.strokeStyle = color;
  ctx.lineWidth = 3;
  ctx.lineCap = 'round';
  ctx.stroke();
  if (emitir) {
    socket.emit('dibujar', { x0, y0, x1, y1, color });
  }
}
canvas.addEventListener('mousedown', (e) => {
  dibujando = true;
  xAnterior = e.offsetX;
  yAnterior = e.offsetY;
});
canvas.addEventListener('mousemove', (e) => {
  if (!dibujando) return;
  dibujarLinea(xAnterior, yAnterior, e.offsetX, e.offsetY, '#e63946', true);
  xAnterior = e.offsetX;
  yAnterior = e.offsetY;
});
```

```
canvas.addEventListener('mouseup', () => { dibujando = false; });
canvas.addEventListener('mouseleave', () => { dibujando = false; });
socket.on('dibujar', (datos) => {
    dibujarLinea(datos.x0, datos.y0, datos.x1, datos.y1, datos.color, false);
});
</script>
</body>
</html>
```

- **El elemento canvas y el contexto 2D**

El elemento HTML **canvas** es una superficie de dibujo en píxeles controlada completamente mediante JavaScript. El método **getContext('2d')** entrega el conjunto de herramientas de dibujo: trazos, figuras, colores y transformaciones. Sin obtener este contexto, el **canvas** es únicamente un rectángulo vacío sin capacidad de renderizar nada.

- **La función dibujarLinea y el parámetro emitir**

Esta función cumple una doble responsabilidad: dibuja el trazo localmente en el **canvas** y, si el parámetro **emitir** es **true**, envía las coordenadas al servidor mediante **socket.emit()**. Cuando el servidor reenvía el evento a los demás clientes y estos lo reciben por **socket.on('dibujar')**, llaman a la misma función con **emitir** en **false**, evitando así un ciclo infinito donde cada cliente reenviaría indefinidamente los trazos recibidos.

- **Los eventos del ratón y el control del estado de dibujo**

La variable **dibujando** actúa como un interruptor: se activa con **mousedown** y se desactiva con **mouseup** o **mouseleave**. Sin el evento **mouseleave**, al arrastrar el cursor fuera del **canvas** y volver a entrar, el servidor continuaría recibiendo coordenadas de dibujo, aunque el usuario no tuviera el botón presionado, generando trazos involuntarios que degradan la experiencia de todos los participantes conectados.

4. Flujo lógico del desarrollador

Construir una aplicación web en tiempo real no es únicamente escribir código correcto, es tomar decisiones técnicas en el orden adecuado. Un desarrollador que salta etapas o que no comprende la secuencia lógica del proceso inevitablemente enfrenta errores difíciles de rastrear, entornos inestables y código difícil de mantener.

El flujo que se describe a continuación sintetiza el recorrido completo abordado en este componente: desde la configuración del entorno hasta la integración de la capa de sockets. No es una secuencia rígida ni definitiva; en la práctica profesional, el desarrollo es iterativo, pero representa el orden lógico recomendado para proyectos de esta naturaleza.

Tabla 3. Flujo lógico del desarrollador para aplicaciones web en tiempo real con Node.js

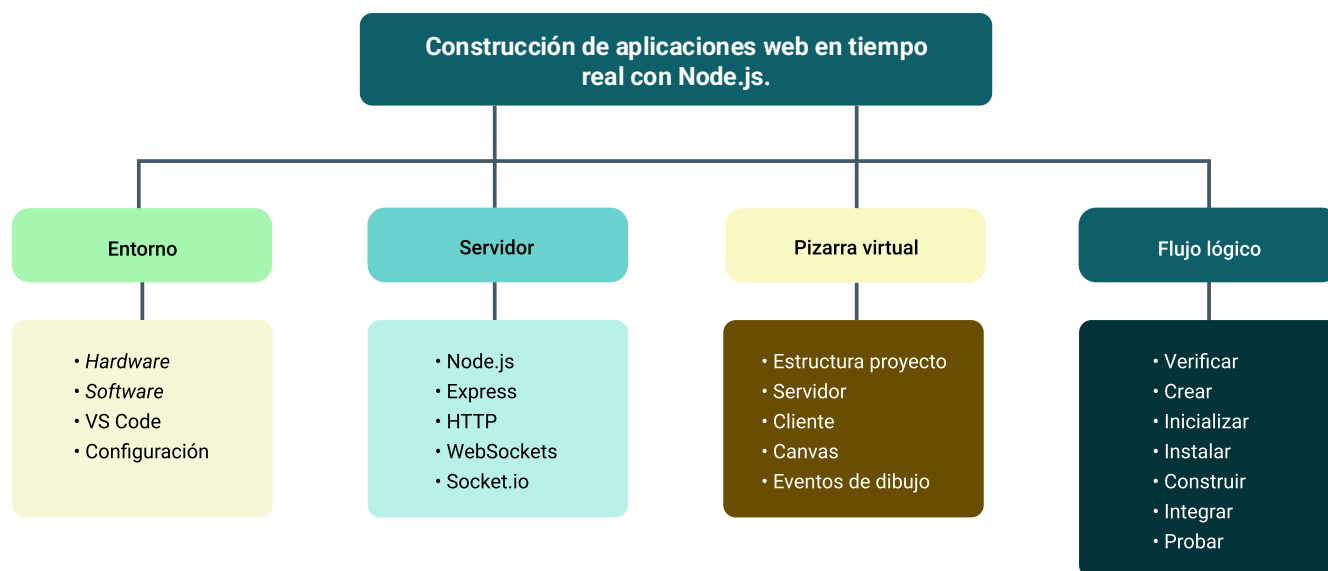
Etapa	Acción	Error frecuente si se omite
1	Verificar versiones de Node.js y npm	Incompatibilidad de dependencias sin origen claro
2	Crear directorio e inicializar con npm init -y	Proyecto sin package.json - dependencias no registradas
3	Instalar Express y socket.io	Módulos no encontrados al ejecutar el servidor
4	Construir el servidor HTTP con Express	Sin base de red no es posible integrar WebSockets
5	Integrar socket.io al servidor	El servidor no puede mantener conexiones persistentes
6	Definir eventos de emisión y escucha	Mensajes que no llegan o se duplican entre clientes

Etapa	Acción	Error frecuente si se omite
7	Construir el cliente HTML con canvas	Sin interfaz no hay forma de probar la comunicación
8	Verificar flujo completo cliente ↔ servidor	Errores silenciosos que solo aparecen con múltiples clientes

Cada etapa depende de la anterior. Intentar integrar sockets sin un servidor HTTP funcional, o construir el cliente antes de definir los eventos del servidor, produce errores de conexión que no indican claramente su origen; el mensaje de error apunta al síntoma, no a la causa.

Síntesis

La construcción de aplicaciones web en tiempo real con Node.js parte de un entorno de desarrollo bien configurado: los componentes físicos (hardware) y el software base (sistema operativo, Node.js, npm y VS Code) son la base sobre la que se inicializa el proyecto mediante **npm init -y** y se instalan las dependencias con **npm install**. Sobre ese entorno se levanta un servidor HTTP con Express, cuyo comportamiento stateless se supera al integrar socket.io: una capa que establece un canal full-duplex persistente mediante un handshake inicial, permitiendo que cliente y servidor intercambien eventos en tiempo real con **socket.emit**, **socket.broadcast.emit** e **io.emit**. Estos conceptos convergen en la pizarra virtual colaborativa, donde la API **canvas** del navegador captura los trazos del usuario, los envía al servidor y este los redistribuye instantáneamente a todos los participantes conectados. El recorrido completo sigue un flujo lógico de ocho etapas: desde la verificación del entorno hasta la prueba de concurrencia con múltiples clientes.



Glosario

API (Interfaz de Programación de Aplicaciones): conjunto de reglas y protocolos que permite la comunicación entre distintas aplicaciones de software. En el contexto del desarrollo web con Node.js, Express facilita la construcción de la API del servidor definiendo las rutas y respuestas disponibles para los clientes.

Broadcast: método de transmisión en socket.io mediante el cual el servidor envía un evento a todos los clientes conectados simultáneamente, con excepción del emisor original. Se implementa con `socket.broadcast.emit`.

Dependencia: librería de código desarrollada por terceros que un proyecto Node.js necesita para funcionar. Se instala con `npm install` y queda registrada en el archivo `package.json`, lo que permite reconstruir el entorno en cualquier equipo.

Electron: marco de trabajo de código abierto que permite desarrollar aplicaciones de escritorio multiplataforma usando tecnologías web (html, css y javascript). Visual Studio Code está construido sobre Electron, lo que explica su comportamiento idéntico en Windows, MacOS y Linux.

Event loop (Bucle de Eventos): mecanismo interno de Node.js que permite ejecutar operaciones de entrada/salida de forma no bloqueante. En lugar de esperar a que una tarea pesada termine, delega su ejecución al sistema operativo y continúa procesando otras solicitudes, lo que posibilita alta concurrencia en un solo hilo.

Express: marco de trabajo minimalista para Node.js que simplifica la creación de servidores HTTP, la definición de rutas y el manejo de peticiones y respuestas. Es el estándar de la industria para construir el servidor base sobre el que se integran capas adicionales como socket.io.

Full-duplex: característica de un canal de comunicación que permite enviar y recibir datos simultáneamente en ambas direcciones sin interrupciones. Es la propiedad fundamental de los websockets que los distingue del protocolo HTTP tradicional, donde solo el cliente puede iniciar la comunicación.

Handshake (Negociación Inicial): proceso automatizado entre cliente y servidor para establecer los parámetros de una conexión WebSocket. El cliente envía una petición HTTP con cabecera de upgrade y, si el servidor la acepta, la conexión HTTP se transforma en un canal TCP persistente y bidireccional.

IDE (Entorno de Desarrollo Integrado): plataforma de software que centraliza en una sola interfaz las herramientas esenciales del desarrollo: editor de código, terminal integrada y depurador. En el ecosistema Node.js, visual studio code es el ide de referencia en la industria.

Localhost: nombre de host que hace referencia a la propia máquina donde se ejecuta el servidor (dirección ip 127.0.0.1). Se utiliza para probar aplicaciones web en el entorno local antes de desplegarlas en un servidor de producción accesible desde internet.

Node.js: entorno de ejecución de javascript de código abierto y multiplataforma, construido sobre el motor v8 de Chrome. Permite ejecutar JavaScript en el servidor, fuera del navegador, y está diseñado para construir aplicaciones de red escalables gracias a su modelo de I/O no bloqueante.

NPM (Node Package Manager): gestor de paquetes oficial de Node.js que permite instalar, actualizar y administrar dependencias de terceros. Incluye acceso al repositorio público de paquetes de código abierto más grande del mundo y se instala automáticamente junto con Node.js.

Package.json: archivo manifiesto ubicado en la raíz de todo proyecto Node.js. Registra el nombre del proyecto, el archivo de entrada principal y el inventario exacto de dependencias instaladas. Permite que cualquier desarrollador reconstruya el entorno completo con un solo comando `npm install`.

Servidor HTTP: software que implementa el protocolo HTTP y es responsable de recibir peticiones de navegadores web y entregar la respuesta correspondiente. En el contexto de Node.js, el servidor HTTP se construye con el módulo nativo HTTP y se organiza mediante el framework `express`.

WebSockets: protocolo de comunicación estandarizado (RFC 6455) que establece un canal persistente, bidireccional y full-duplex sobre una única conexión TCP. A diferencia del protocolo HTTP, permite que el servidor envíe datos al cliente en cualquier momento sin esperar una petición previa, lo que lo hace ideal para chats, tableros y aplicaciones colaborativas en tiempo real.

Referencias bibliográficas

[B](#)rown, E. (2020). Web development with Node and Express: Leveraging the JavaScript stack (2.^a ed.). O'Reilly Media.

MDN Web Docs. (2023). Canvas API. Mozilla Foundation.

MDN Web Docs. (2023). WebSockets API. Mozilla Foundation.

Microsoft. (2024). Node.js tutorial in Visual Studio Code. Microsoft.

Node.js Foundation. (2024). Introduction to Node.js. OpenJS Foundation.

Npm. (2024). About npm. npm, Inc.

Socket.IO. (2024). Introduction to Socket.IO. Socket.IO.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Edison Eduardo Mantilla Cuadros	Responsable de línea de producción	Centro Agroturístico - Regional Santander
Carlos Andrés Bonza Reyes	Experto temático	Centro Agroturístico - Regional Santander
Angélica Varón Quintero	Evaluadora instruccional	Centro Agroturístico - Regional Santander
Marcos Yamid Rubiano Avellaneda	Diseñador de contenidos	Centro Agroturístico - Regional Santander
Lizeth Karina Manchego Suarez	Desarrolladora full stack	Centro Agroturístico - Regional Santander
María Alejandra Vera Briceño	Animadora y productora audiovisual	Centro Agroturístico - Regional Santander
Laura Paola Gelvéz Manosalva	Validadora de recursos educativos digitales	Centro Agroturístico - Regional Santander
Sandra Liliana Cristancho Cruz	Evaluadora de contenidos inclusivos y accesibles	Centro Agroturístico - Regional Santander